Lab 2 Report

Chen Liangkun 23020036007. Lab group: 4.

Lab date: 11th November 2025. Report date: 22nd March 2026.

1 Lab Introduction and Team Information

Install ROS, get familiar with it.

2 Individual Work

2.1 Install ROS

To install ROS, we need to add the source of ROS to apt and trust it. This part is quite easy so we just show the command.

```
sudo sh -c 'echo deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main
    > /etc/apt/sources.list.d/ros-latest.list'
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo
   apt-key add -
sudo apt update
sudo apt install ros-noetic-desktop-full
```

Then we set the environment var and install the extension dependency. After all is done, we do the initialization.

```
echo source /opt/ros/noetic/setup.bash >> ~/.bashrc
source ~/.bashrc
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator
    python3-vcstool build-essential python3-catkin-tools python-is-python3sudo
    rosdep init
rosdep update
iangkun@ubuntu:~/vnav-personal/lab1/RandomVector$ sudo rosdep init
Wrote /etc/ros/rosdep/sources.list.d/20-default.list
Recommended: please run

        rosdep update

liangkun@ubuntu:~/vnav-personal/lab1/RandomVector$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.
   yaml
```

```
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://raw.githubusercontent.com/ros/rosdistro/master/
    index-v4.yaml
Skip end-of-life distro ardent
Skip end-of-life distro bouncy
Skip end-of-life distro crystal
Skip end-of-life distro dashing
Skip end-of-life distro eloquent
Skip end-of-life distro foxy
Skip end-of-life distro galactic
Skip end-of-life distro groovy
Add distro humble
Skip end-of-life distro hydro
Skip end-of-life distro indigo
Skip end-of-life distro iron
Skip end-of-life distro jade
Add distro jazzy
Add distro kilted
Skip end-of-life distro kinetic
Skip end-of-life distro lunar
Skip end-of-life distro melodic
Skip end-of-life distro noetic
Add distro rolling
updated cache in /home/liangkun/.ros/rosdep/sources.cache
```

2.2 The fundamentals of ROS

In this section, we will go to see how the ROS operate on our machine. We need to start ROS master first

```
liangkun@ubuntu:~/Desktop$ roscore
... logging to /home/liangkun/.ros/log/3ab69228-becb-11f0-81d6-838f79381de3/
    roslaunch-ubuntu-31400.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu:36107/
ros_comm version 1.17.4


SUMMARY
========

PARAMETERS
 * /rosdistro: noetic
 * /rosversion: 1.17.4
```

```
NODES

auto-starting new master
process[master]: started with pid [31411]
ROS_MASTER_URI=http://ubuntu:11311/

setting /run_id to 3ab69228-becb-11f0-81d6-838f79381de3
process[rosout-1]: started with pid [31421]
started core service [/rosout]
```

The basis of ROS is the node, so we will try to create a node in the terminal first. We need a ROS master to manage the nodes' publishing and subscribing. So two terminals are needed, one for the master and one for the node.

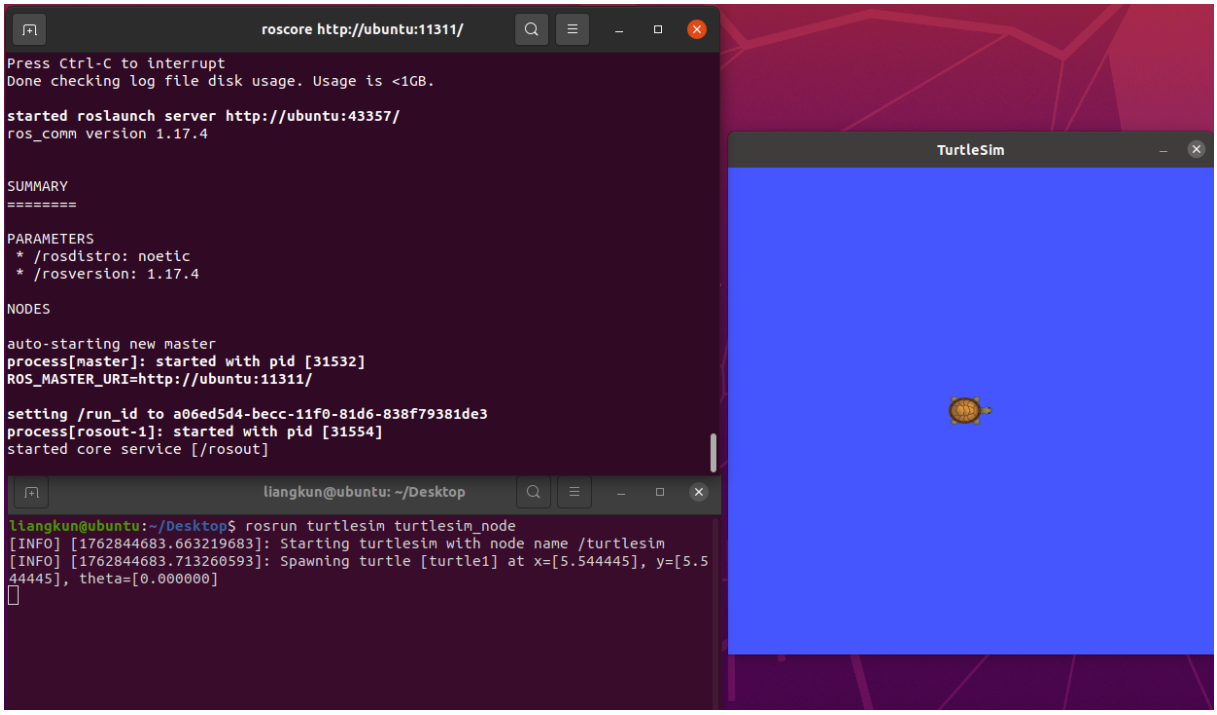


Figure 1: Turtle

We can also check the node now running. And try to add node to see the change of the rosnode.

```
liangkun@ubuntu:~/Desktop$ rosnode list
/rosout
/turtlesim

rosrun turtlesim turtlesim_node

liangkun@ubuntu:~/Desktop$ rosnode list
/rosout
/teleop_turtle
/turtlesim
```

A further check would be the node and the topic. We can show them in a graphical way. Just use the command below and you will see

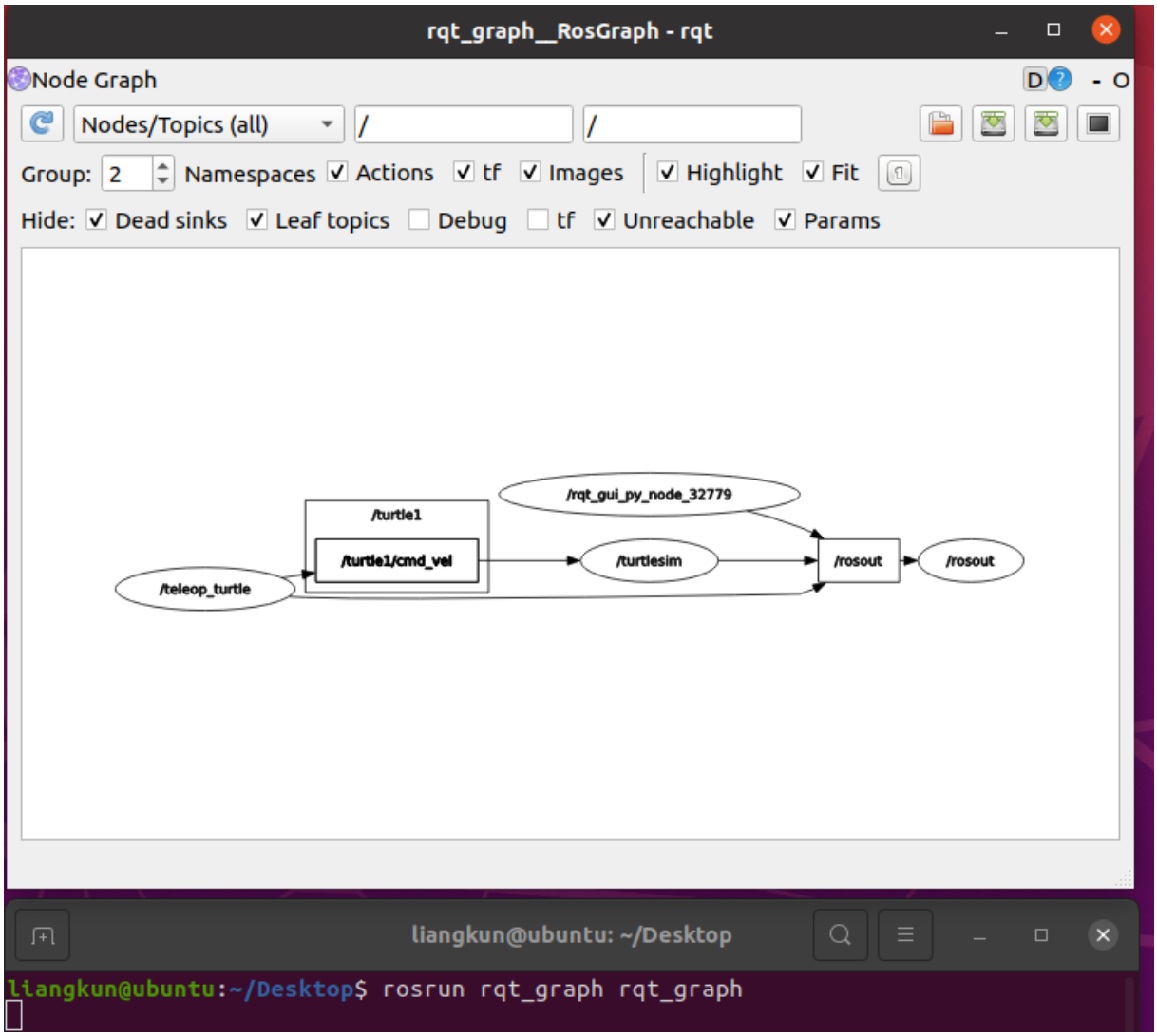


Figure 2: The nodes and topics

In the graph the ellipses are nodes and the squares are topics. From the picture it's easy to see that teleop_turtle is publishing to /turtle1/cmd_vel topic. The node /turtlesim is subscribed to the topic and uses the incoming messages to move the turtle. Let's print it in the terminal

```
liangkun@ubuntu:~/Desktop$ rostopic echo /turtle1/cmd_vel
linear:
  x: -2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

```
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -2.0
---
```

Here we move the turtle forward and back, then we turn a specific angular.

Pretty funny. Now let's move to C++ code.

```
#include <ros/ros.h> //Add the lib

int main(int argc, char **argv) {
  ros::init(argc, argv, example_node); //Initial the node
  //Give node its name and receive param from terminal
  ros::NodeHandle n; //Contact with ROS master.

  ros::Rate loop_rate(50);//Set frequency

  while (ros::ok()) {//Turn false when node is shutting down
    // ... do some useful things ...
    ros::spinOnce();
    loop_rate.sleep();
  }
  return 0;
}
```

Launch files are the preferred way to run ROS nodes. The launch files provide a convenient interface to execute multiple nodes and a master (if is not already running), as well as other initialization requirements such as parameters. We use it to open turtle with a single terminal.

```
roslaunch turtle_tf turtle_tf_demo.launch
```

The rqt_tf_tree tool enables the real-time visualization of the tree of frames being broadcast over ROS. The tf_echo tool reports the transformation between any two frames broadcast over ROS. And RViz is a graphical 3D visualization tool that is useful for viewing the association between TF frames within the ROS system. All of this will be used in the task under.

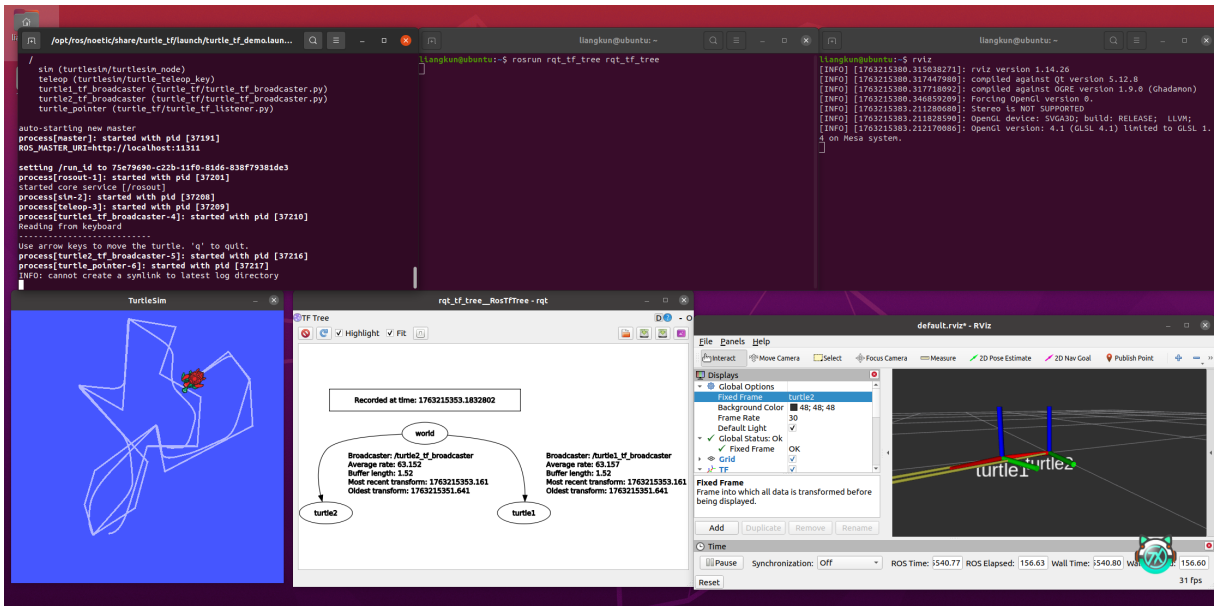


Figure 3: We will see them later

2.3 Use TF to control drone

First, we initialize the catkin

We then copy two_drones_pkg to the folder of src. And compile it. Remember we need to index the compiled file to the terminal. Now we should see two drones as the code construct. Then we can add the axis by adding TF.

From the part 2 we can give out several answer to bellowed questions.

1. Lists the nodes (nodes) running in the static scenario of the two drones.

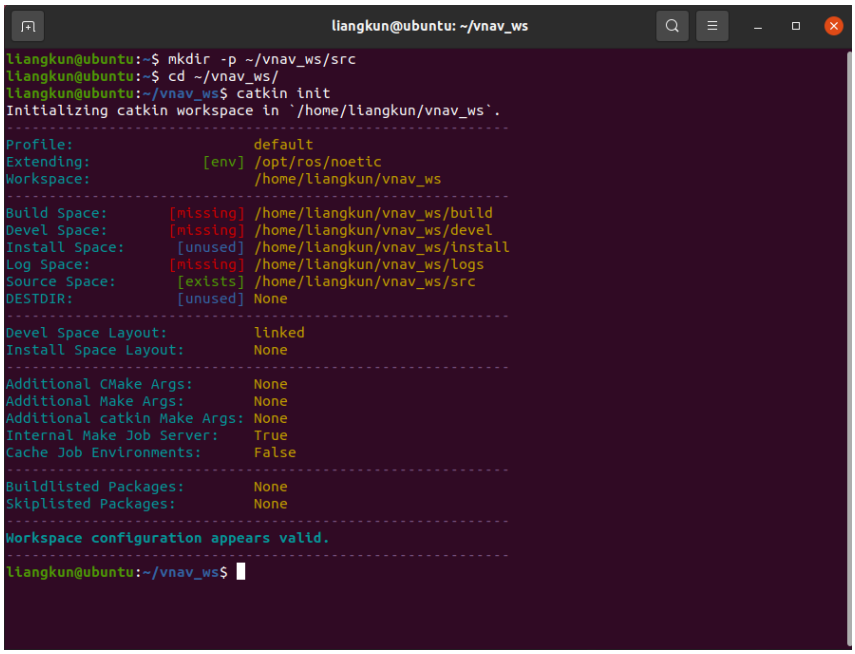
   Ans: We can try two ways, one is the launch file and one is the rpt_graph. We first try launch. The launch file is one of the way run the node but is more preferred in ROS. So it will also describe the important messages about the nodes.

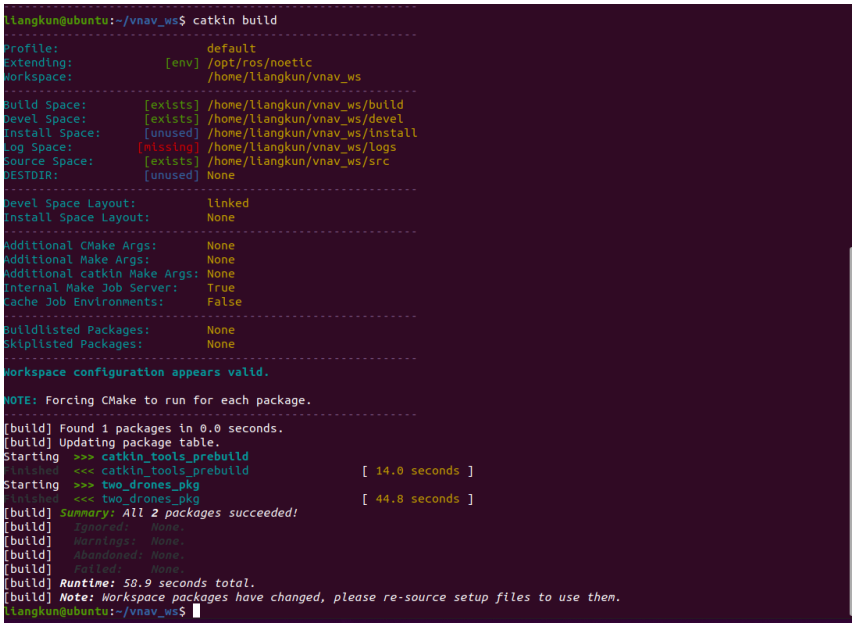
```
 liangkun@ubuntu:~/vnav_ws/src/two_drones_pkg/launch$ cat two_drones.
launch

 <launch>
   <arg name=static default=false/>

   <!-- Transform publishers !-->
   <group if=$(arg static)>
     <node pkg=tf2_ros type=static_transform_publisher name=
av1broadcaster args=1 0 0 0 0 0 1 world av1/>
     <node pkg=tf2_ros type=static_transform_publisher name=
av2broadcaster args=0 0 1 0 0 0 1 world av2/>
   </group>

   <node name=frames_publisher_node pkg=two_drones_pkg type=
frames_publisher_node unless=$(arg static)/>
```





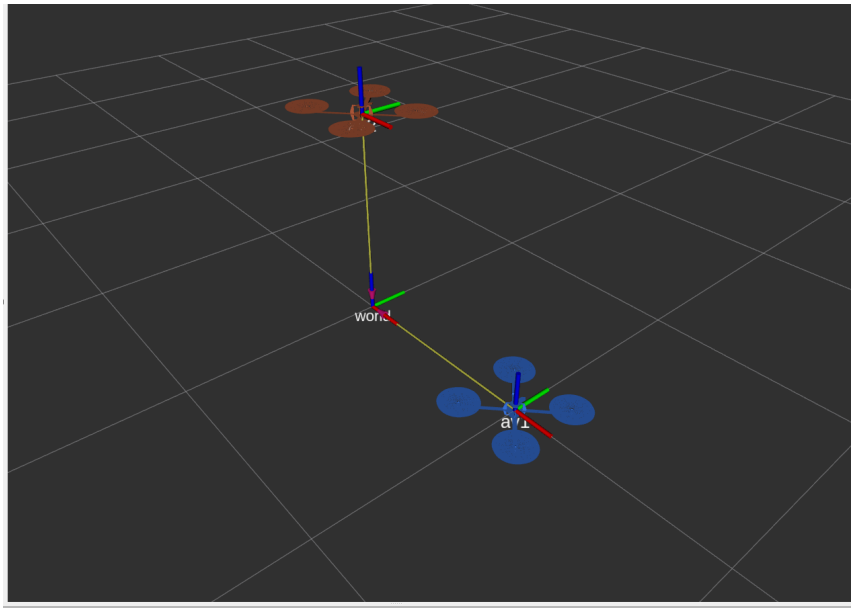
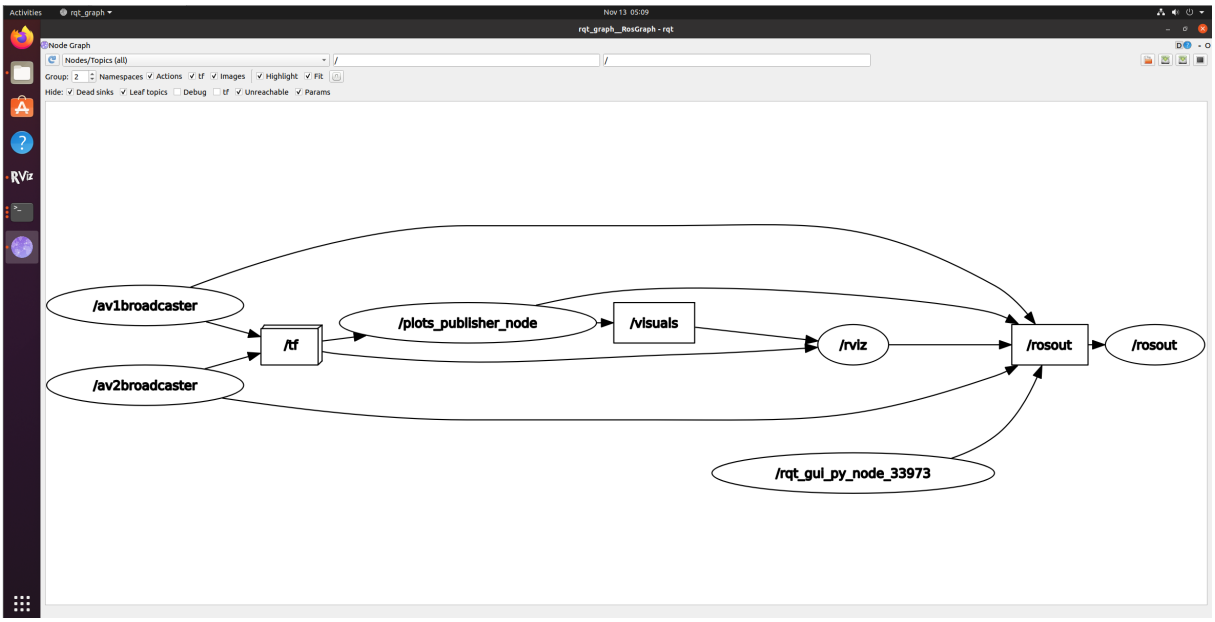


Figure 4: The drones




```
  <!-- Marker publisher -->
  <node name=plots_publisher_node pkg=two_drones_pkg type=
plots_publisher_node/>

  <!-- Visualizer -->
  <node name=rviz pkg=rviz type=rviz args=-d $(find two_drones_pkg)/
config/default.rviz/>
 </launch>
```

Therefore we see the node that from tf2_ros. These TFs will cause the two drones to appear in different locations in RViz. frames_publisher_node will running only when static is false, but this is a static scene, so we this node is not running. Next we see plots_publisher_node and rviz that will run without condition. So actually there are(include master)

- av1broadcaster
- av2broadcaster
- plots_publisher_node
- rviz
- rosout

The figure below shall prove this.

2. How can I run a static scene of two drones without using the roslaunch command?

   Ans: `rosrun` command can run the node manually. We just need to use rosrun to run the node in the launch file. It's equivalent.

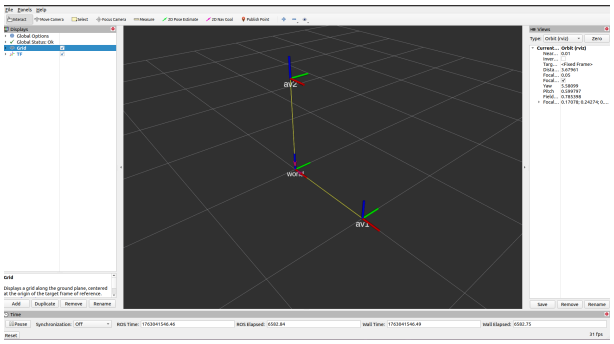


Figure 5: Without Visuals

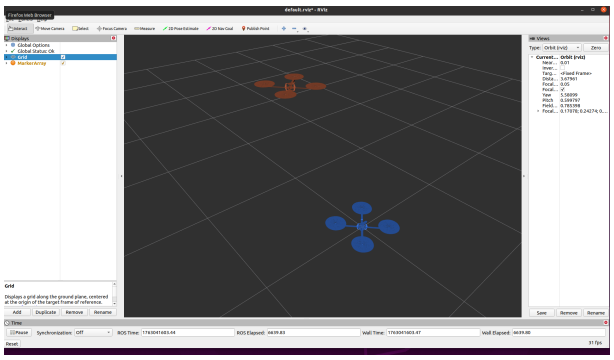


Figure 6: Without TF

3. List the publishes/subscribes topics for each node. Which nodes are responsible for publishing av1, av2, and frames? Which theme is used to draw drone meshes in rViz?

   Ans: We can back to the figure in Qes 2 for the answer.

   - av1broadcaster: Its publishes topics none. Its subscribes topics are `/tf` and `/rosout`
   - av2broadcaster: Same with av1broadcaster.
   - plots_publisher_node: Its publishes topic is . Its subscribes topics are `/visuals` and `/rosout`
   - rviz: Its publishes topics are `/tf` and `/visuals`. Its subscribes topic is `/rosout`

   av1broadcaster and av2broadcaster are responsible for publishing av1, av2, and frames and /Visuals is responsible for drone meshes drawing.

4. What changes if we omit static: =True? Why?

   Ans: Just like we mentioned upward. The static will be false when default. "if" means only if the condition is true the group or the node will run. "unless" is the opposite , only if the condition is false the group or node will run. So if the static is omitted then the node frames_publisher_node will run. But as we haven't implement it yet so we can't see anything.

2.4 Move

Now, we can move to frames_publisher_node file to implement it. We first need to record the time so the tf tree can calculate trajectory.

```
ros:: Time now = ros::Time::now();
double time = (now - startup_time).toSec();
```

What we should do next is to assign the TransformStamped.

```
    AV1World.header.frame_id = world;
    AV1World.header.stamp = now;
    AV1World.child_frame_id = av1;
    AV1World.transform.translation.x = cos(time);
    AV1World.transform.translation.y = sin(time);
    AV1World.transform.translation.z = 0.0;
    tf2::Quaternion q;
    q.setRPY(0.0, 0.0, time);
    AV1World.transform.rotation.x = q.x();
    AV1World.transform.rotation.y = q.y();
    AV1World.transform.rotation.z = q.z();
    AV1World.transform.rotation.w = q.w();

    AV2World.header.frame_id = world;
    AV2World.header.stamp = now;
    AV2World.child_frame_id = av2;
    AV2World.transform.translation.x = sin(time);
    AV2World.transform.translation.y = 0.0;
    AV2World.transform.translation.z = cos(2*time);
    AV2World.transform.rotation.x = 0.0;
    AV2World.transform.rotation.y = 0.0;
    AV2World.transform.rotation.z = 0.0;
    AV2World.transform.rotation.w = 1.0;
```

Here AVxWorld means how the homogeneous transformation matrix at a specific point in time between AVx and world. To avoid gimbal lock we prefer to use Quaternion. Because AV1 is rotating so we transfer Euler angle to quaternion . As this rotate can be equal to rotating through z. So we transfer [0 0 time] to setRPY. Because parameter w is given so we now can pass translation and rotation to AV1World and the homogeneous transformation matrix is finished. AV2World is simpler as it doesn't need to care rotation.

Now the only thing we need to do is broadcast the matrix to tf tree. We initialize a transform broadcaster and call its method.

```
tf2_ros::TransformBroadcaster br;

br.sendTransform(AV1World);
br.sendTransform(AV2World);
```

To see the video please visit here.

2.5 Looking up a transform

We also want to fill the transform with the most recent transform.

```
try {
    // ~~~~~~~~~~~~~~~~~~~~~~  BEGIN OF EDIT SECTION  ~~~~~~~~~~~~~~~~~~~~~~~~~

    /* The transform object needs to be populated with the most recent
     * transform from ref_frame to dest_frame as provided by tf.

     * Relevant variables in this scope:
     *   - ref_frame, the frame of reference relative to which the trajectory
     *                needs to be plotted (given)
     *   - dest_frame, the frame of reference of the object whose trajectory
     *                 needs to be plotted (given)
     *   - parent->tf_buffer, a tf2_ros::Buffer object (given)
     *   - transform, the geometry_msgs::TransformStamped object that needs to
    be populated
     *
     * HINT: use lookupTransform, see
     * https://wiki.ros.org/tf2/Tutorials/Writing%20a%20tf2%20listener%20%28C%2
    B%2B%29#TheCode
     */

    transform = parent->tf_buffer.lookupTransform(ref_frame, dest_frame, ros::
    Time(0));


    // ~~~~~~~~~~~~~~~~~~~~~~~  END OF EDIT SECTION  ~~~~~~~~~~~~~~~~~~~~~~~~~
    while (poses.size() >= buffer_size) {
      poses.pop_front();
    }

    geometry_msgs::Point tmp;
    tmp.x = transform.transform.translation.x;
    tmp.y = transform.transform.translation.y;
    tmp.z = transform.transform.translation.z;
    poses.push_back(tmp);
    } catch (const tf2::TransformException& ex) {
    ROS_ERROR_STREAM(transform lookup failed:  << ex.what());
    }
```

We want the transform to this frame (ref_frame) from this frame (dest_frame). The time at which we want to transform. Providing ros::Time(0) will just get us the latest available transform. You can see video here

2.6 Math

1. In the problem formulation ,we mentioned that AV2's trajectory is an arc of parabola in the x-$z$plane of the world frame. Can you prove this statement?

Ans:

$$sin^2(t) = \frac{1 - cos(2t)}{2}, \tag{1}$$

let $sin(t) = z$, $cos(t) = x$ we have

$$z^2 = \frac{1}{2} - \frac{x}{2}. \tag{2}$$

This is a an arc of parabola.

2. Compute $o_2^1(t)$, i.e., the position of AV2 relative to AV1's body frame as a function of t .

$$T_w^2 = \begin{bmatrix} 1 & 0 & 0 & sin(t) \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & cos(2t) \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{3}$$

$$T_w^1 = \begin{bmatrix} cos(t) & -sin(t) & 0 & cos(t) \\ sin(t) & cos(t) & 0 & sin(t) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{4}$$

$$o_2^1 = T_w^1 \times T_2^w \times o_2, \tag{5}$$

$$o_2^1 = \begin{bmatrix} cos(t)sin(t) - 1 \\ -sin^2(t) \\ cos(2t) \end{bmatrix}. \tag{6}$$

3. Show that $o_2^1(t)$ describes a planar curve and find the equation of its plane Π.

   Ans:

$$cos(2t) = 1 - 2sin^2(t), \tag{7}$$

   So that is

$$z = 1 + 2y \tag{8}$$

   This is the plane where o_2^1 at.

4. Rewrite the above trajectory explicitly using a 2D frame of reference (x_p, y_p) on the plane found before. Try to ensure that the curve is centered at the origin of this 2D frame and that x_p , y_p are axes of symmetry for the curve.

   Ans: We need to find the transform matrix from p to 1 first. That is

$$T_p^1 = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & \frac{1}{\sqrt{5}} & -\frac{2}{\sqrt{5}} & -\frac{1}{2} \\ 0 & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{9}$$

   We calculate its reverse which is

$$T_1^p = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & \frac{1}{\sqrt{5}} & \frac{2}{\sqrt{5}} & \frac{1}{2\sqrt{5}} \\ 0 & -\frac{2}{\sqrt{5}} & \frac{1}{\sqrt{5}} & -\frac{1}{\sqrt{5}} \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{10}$$

Then we calculate the o_2^p

$$o_2^p = T_1^p \times o_2^1 = \begin{bmatrix} \frac{sin(2t)}{2} \\ \frac{\sqrt{5}}{2} cos(2t) \\ 0 \end{bmatrix}, \tag{11}$$

This can be transform to

$$\frac{x^2}{\frac{1}{4}} + \frac{y^2}{\frac{5}{4}} = 1. \tag{12}$$

Which is obviously an oval. An oval is centered at the origin of this 2D frame and that x_p, y_p are axes of symmetry for the curve.

5. Using the expression of $o_2^p(t)$, prove that the trajectory of AV2 relative to AV1 is an ellipse and compute the lengths of its semi-axes.

   Ans: This has been proven upward. And from the expression of the ellipse in equation 12, we can easily calculate the semi-axes' length.

$$b^2 = \frac{5}{4}, \tag{13}$$

$$b = \frac{\sqrt{5}}{2}. \tag{14}$$

2.7 More properties of quaternions

We want to prove some interesting properties about quaternions. We have defined two linear maps $\Omega_1 : R^4 \to R^{4\times4}$ and $\Omega_2 : R^4 \to R^{4\times4}$, such that for any $q \in R^4$, we have:

$$\Omega_1(q) = \begin{bmatrix} q_4 & -q_3 & q_2 & q_1 \\ q_3 & q_4 & -q_1 & q_2 \\ -q_2 & q_1 & q_4 & q_3 \\ -q_1 & -q_2 & -q_3 & q_4 \end{bmatrix}, \ \Omega_2(q) = \begin{bmatrix} q_4 & q_3 & -q_2 & q_1 \\ -q_3 & q_4 & q_1 & q_2 \\ q_2 & -q_1 & q_4 & q_3 \\ -q_1 & -q_2 & -q_3 & q_4 \end{bmatrix}.$$

The product between any two unit quaternions can then be explicitly computed as:

$q_a \otimes q_b = \Omega_1(q_a)q_b = \Omega_2(q_b)q_a$

In fact, the two linear maps Ω_1 and Ω_2 have more interesting properties, and you are asked to prove the following equalities:

1. For any unit quaternion q, both $\Omega_1(q)$ and $\Omega_2(q)$ are orthogonal matrices, i.e.,

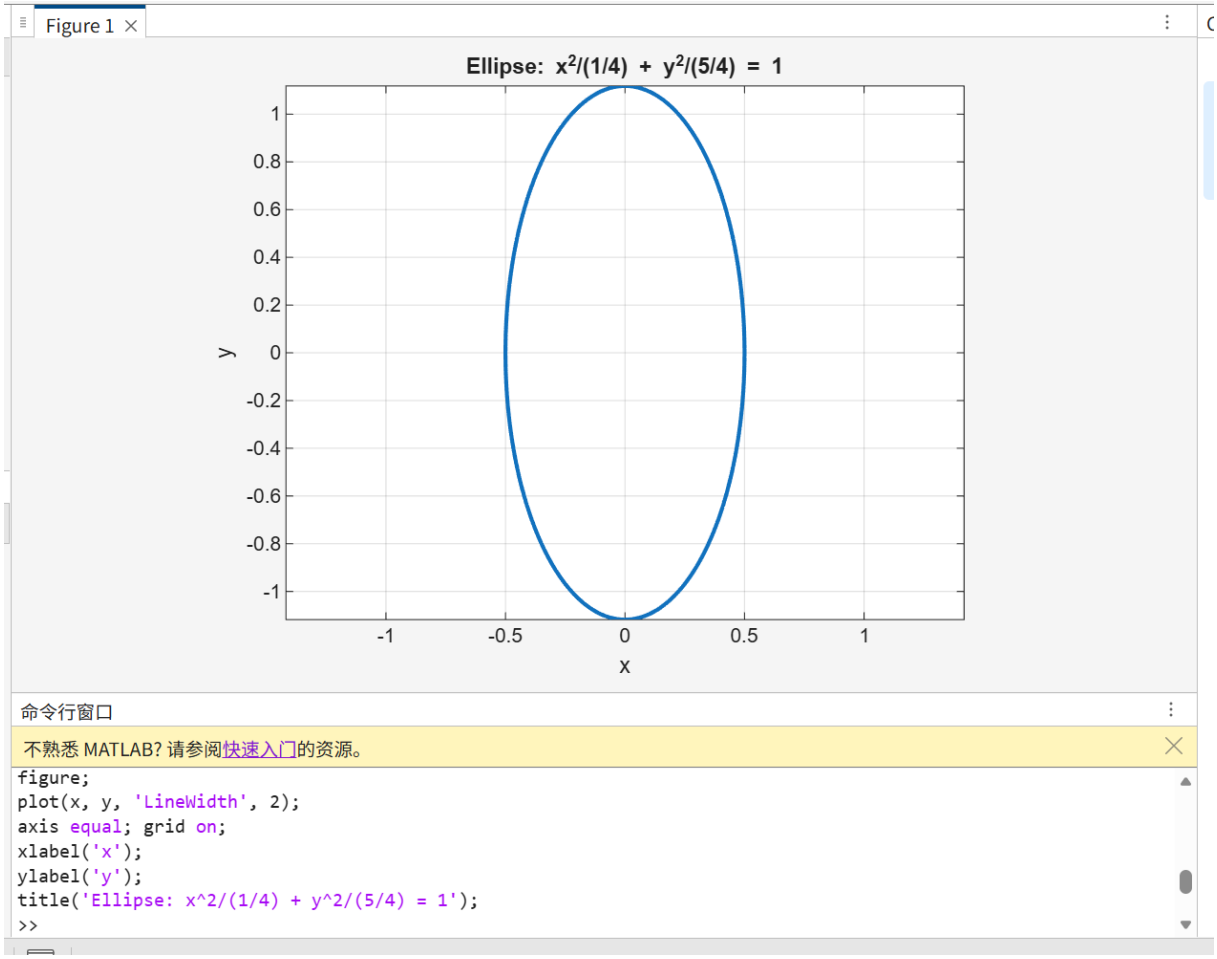
$$\Omega_1(q)^T\Omega_1(q) = \Omega_1(q)\Omega_1(q)^T = I_4, \tag{15}$$

$$\Omega_2(q)^T\Omega_2(q) = \Omega_2(q)\Omega_2(q)^T = I_4. \tag{16}$$

   Intuitively, why must both of them be orthogonal? - Hint: what does $q_a \otimes q_b$ still being a unit quaternion imply?

   ANS: We do the calculation

$$\Omega_1(q)\Omega_1(q)^T = \begin{bmatrix} q_4 & -q_3 & q_2 & q_1 \\ q_3 & q_4 & -q_1 & q_2 \\ -q_2 & q_1 & q_4 & q_3 \\ -q_1 & -q_2 & -q_3 & q_4 \end{bmatrix} \begin{bmatrix} q_4 & q_3 & -q_2 & -q_1 \\ -q_3 & q_4 & q_1 & -q_2 \\ q_2 & -q_1 & q_4 & -q_3 \\ q_1 & q_2 & q_3 & q_4 \end{bmatrix}, \tag{17}$$



$$||q|| = 1, \tag{18}$$

$$\Omega_1(q)^T\Omega_1(q) = I_4. \tag{19}$$

This is the same on the linear map Ω_2.

$$\Omega_2(p)\Omega_2^T(p) = \begin{bmatrix} q_4 & q_3 & -q_2 & q_1 \\ -q_3 & q_4 & q_1 & q_2 \\ q_2 & -q_1 & q_4 & q_3 \\ -q_1 & -q_2 & -q_3 & q_4 \end{bmatrix} \begin{bmatrix} q_4 & -q_3 & q_2 & -q_1 \\ q_3 & q_4 & -q_1 & -q_2 \\ -q_2 & q_1 & q_4 & -q_3 \\ q_1 & q_2 & q_3 & q_4 \end{bmatrix} \tag{20}$$

With equation 18, we have

$$\Omega_2(q)^T\Omega_2(q) = I_4. \tag{21}$$

We can use MATLAB to prove this

```
        >>
Omega1 = [ q4, -q3, q2, q1;
           q3, q4, -q1, q2;
          -q2, q1, q4, q3;
          -q1, -q2, -q3, q4];

%
Omega2 = [ q4, q3, -q2, q1;
          -q3, q4, q1, q2;
```

```
           q2, -q1, q4, q3;
          -q1, -q2, -q3, q4];
>>
Prod1 = transpose(Omega1) * Omega1;


SimplifiedProd1 = simplify(Prod1);


IdentityMatrix = eye(4);
Difference1 = SimplifiedProd1 - IdentityMatrix;



disp(SimplifiedProd1);

[q1^2 + q2^2 + q3^2 + q4^2,                          0,
        0,                          0]
[                        0, q1^2 + q2^2 + q3^2 + q4^2,
        0,                          0]
[                        0,                          0, q1^2 + q2^2 + q3^2
   + q4^2,                          0]
[                        0,                          0,
        0, q1^2 + q2^2 + q3^2 + q4^2]

>>
Prod2 = transpose(Omega2) * Omega2;
SimplifiedProd2 = simplify(Prod2);



disp(SimplifiedProd2);

[q1^2 + q2^2 + q3^2 + q4^2,                          0,
        0,                          0]
[                        0, q1^2 + q2^2 + q3^2 + q4^2,
        0,                          0]
[                        0,                          0, q1^2 + q2^2 + q3^2
   + q4^2,                          0]
[                        0,                          0,
        0, q1^2 + q2^2 + q3^2 + q4^2]

>>
```

The fundamental reason that $\Omega_1(q)$ and $\Omega_2(q)$ must be orthogonal for a unit quaternion q is related to the preservation of the quaternion norm under multiplication.The norm of the product of two quaternions is the product of their individual norms:

$$\|q_a \otimes q_b\| = \|q_a\|\|q_b\|$$

If q_a and q_b are unit quaternions, then $\|q_a\| = 1$ and $\|q_b\| = 1$. This implies that

their product, $q_a \otimes q_b$, must still be a unit quaternion:

$$\|q_a \otimes q_b\| = 1 \cdot 1 = 1$$

In matrix-vector form, the product is $q_a \otimes q_b = \Omega_1(q_a)q_b$. The preservation of the norm is:

$$\|\Omega_1(q_a)q_b\|^2 = \|q_b\|^2$$
$$(\Omega_1(q_a)q_b)^T(\Omega_1(q_a)q_b) = q_b^T q_b$$
$$q_b^T\Omega_1(q_a)^T\Omega_1(q_a)q_b = q_b^T I_4 q_b$$

Since this must hold for any unit quaternion q_b, and we know q_a is a unit quaternion, the matrix $\Omega_1(q_a)$ must be an isometry (a transformation that preserves length/norm). In R^4, linear transformations that preserve the norm are precisely the orthogonal matrices, i.e., $\Omega_1(q_a)^T\Omega_1(q_a) = I_4$. The same logic applies to $\Omega_2(q)$.

2. 2. For any unit quaternion q, both $\Omega_1(q)$ and $\Omega_2(q)$ convert q to be the unit quaternion that corresponds to the 3D identity rotation, *i.e.* $\Omega_1(q)^T q = \Omega_2(q)^T q = [0, 0, 0, 1]^T$.

   Ans:

$$\Omega_1^T(p)p = \begin{bmatrix} q_4 & q_3 & -q_2 & -q_1 \\ -q_3 & q_4 & q_1 & -q_2 \\ q_2 & -q_1 & q_4 & -q_3 \\ q_1 & q_2 & q_3 & q_4 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}^T \tag{22}$$

$$\Omega_2^T(p)p = \begin{bmatrix} q_4 & -q_3 & q_2 & -q_1 \\ q_3 & q_4 & -q_1 & -q_2 \\ -q_2 & q_1 & q_4 & -q_3 \\ q_1 & q_2 & q_3 & q_4 \end{bmatrix} \begin{bmatrix} p1 \\ p2 \\ p3 \\ p4 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \tag{23}$$

   Proved

3. 3. For any two vectors $x, y \in R^4$, show the two linear operators commute, *i.e.*

   $\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x)$,

   $\Omega_1(x)\Omega_2(y)^T = \Omega_2(y)^T\Omega_1(x)$.

   Ans: The commutation of the operators is equivalent to the associativity of quaternion multiplication.The expression $\Omega_1(x)\Omega_2(y)$ operating on a vector $z \in R^4$ is:

$$(\Omega_1(x)\Omega_2(y))z = \Omega_1(x)(\Omega_2(y)z)$$

   Using the definitions of Ω_1 (left-multiplication) and Ω_2 (right-multiplication):

$$\Omega_1(x)(\Omega_2(y)z) = \Omega_1(x)(z \otimes y) = x \otimes (z \otimes y)$$

   The expression $\Omega_2(y)\Omega_1(x)$ operating on a vector $z \in R^4$ is:

$$(\Omega_2(y)\Omega_1(x))z = \Omega_2(y)(\Omega_1(x)z)$$

   Using the definitions of Ω_1 and Ω_2:

$$\Omega_2(y)(\Omega_1(x)z) = \Omega_2(y)(x \otimes z) = (x \otimes z) \otimes y$$

For the matrices to commute, we must have:

$$\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x)$$

Which means, for any z:

$$x \otimes (z \otimes y) = (x \otimes z) \otimes y$$

This is the property of associativity for quaternion multiplication, which holds true for all $x, y, z \in R^4$. Thus, the first commutation relation $\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x)$ is satisfied.For the second commutation relation, $\Omega_1(x)\Omega_2(y)^T = \Omega_2(y)^T\Omega_1(x)$:We use the result from part 2 that $\Omega_2(y)^T = \Omega_2(y^*)$.

$$\Omega_1(x)\Omega_2(y)^T = \Omega_1(x)\Omega_2(y^*)$$

Operating on z:

$$\Omega_1(x)(\Omega_2(y^*)z) = \Omega_1(x)(z \otimes y^*) = x \otimes (z \otimes y^*)$$

And:

$$\Omega_2(y)^T\Omega_1(x) = \Omega_2(y^*)\Omega_1(x)$$

Operating on z:

$$\Omega_2(y^*)(\Omega_1(x)z) = \Omega_2(y^*)(x \otimes z) = (x \otimes z) \otimes y^*$$

Since quaternion multiplication is associative, $x \otimes (z \otimes y^*) = (x \otimes z) \otimes y^*$, the second commutation relation $\Omega_1(x)\Omega_2(y)^T = \Omega_2(y)^T\Omega_1(x)$ is also satisfied.The commutation of the operators is equivalent to the associativity of quaternion multiplication.The expression $\Omega_1(x)\Omega_2(y)$ operating on a vector $z \in R^4$ is:

$$(\Omega_1(x)\Omega_2(y))z = \Omega_1(x)(\Omega_2(y)z)$$

Using the definitions of Ω_1 (left-multiplication) and Ω_2 (right-multiplication):

$$\Omega_1(x)(\Omega_2(y)z) = \Omega_1(x)(z \otimes y) = x \otimes (z \otimes y)$$

The expression $\Omega_2(y)\Omega_1(x)$ operating on a vector $z \in R^4$ is:

$$(\Omega_2(y)\Omega_1(x))z = \Omega_2(y)(\Omega_1(x)z)$$

Using the definitions of Ω_1 and Ω_2:

$$\Omega_2(y)(\Omega_1(x)z) = \Omega_2(y)(x \otimes z) = (x \otimes z) \otimes y$$

For the matrices to commute, we must have:

$$\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x)$$

Which means, for any z:

$$x \otimes (z \otimes y) = (x \otimes z) \otimes y$$

This is the property of associativity for quaternion multiplication, which holds true for all $x, y, z \in R^4$. Thus, the first commutation relation $\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x)$ is satisfied.For the second commutation relation, $\Omega_1(x)\Omega_2(y)^T = \Omega_2(y)^T\Omega_1(x)$:We use the result from part 2 that $\Omega_2(y)^T = \Omega_2(y^*)$.

$$\Omega_1(x)\Omega_2(y)^T = \Omega_1(x)\Omega_2(y^*)$$

Operating on z:

$$\Omega_1(x)(\Omega_2(y^*)z) = \Omega_1(x)(z \otimes y^*) = x \otimes (z \otimes y^*)$$

And:

$$\Omega_2(y)^T\Omega_1(x) = \Omega_2(y^*)\Omega_1(x)$$

Operating on z:

$$\Omega_2(y^*)(\Omega_1(x)z) = \Omega_2(y^*)(x \otimes z) = (x \otimes z) \otimes y^*$$

Since quaternion multiplication is associative, $x \otimes (z \otimes y^*) = (x \otimes z) \otimes y^*$, the second commutation relation $\Omega_1(x)\Omega_2(y)^T = \Omega_2(y)^T\Omega_1(x)$ is also satisfied.

3 Discussion

The Part 4 is the math analysis of the part 3. This shows how the transform matrix work on the problem. The part 5 shows the advantage of the quaternion.

4 Conclusions

This Lab introduces the ROS. We have a basic feeling to the ROS and we do the math part to get to know the basic calculation in the ROS. And get understand of how it work to calculate the pose of the robot.

References